ShopSwift

E-Commerce Application

Full-Stack Technical Documentation

Node.js · Express · MongoDB · React · Vite · Context API

Version 1.0 · Academic Project · 2025

4 Build Stages · 28 Files

4	5	28	4
Build Stages	API Endpoints	Source Files	Categories

ShopSwift is a full-stack e-commerce web application built as an academic project. It demonstrates product browsing, cart management, and a simulated checkout flow using a modern React frontend and a RESTful Node.js/MongoDB backend.

TABLE OF CONTENTS

Contents

01 PROJECT OVERVIEW

Project Overview

ShopSwift is a full-stack e-commerce web application developed as an academic e-business project. It demonstrates the core features of a modern online store — product browsing, category filtering, a persistent shopping cart, and a multi-step simulated checkout — without requiring real payment integration or user authentication.

1.1 Purpose & Scope

The application is designed to showcase:

- A clean RESTful API built with Node.js and Express, backed by MongoDB
- A modern React frontend using Vite, React Router, and Context API
- Separation of concerns through an MVC backend and layered frontend architecture
- Realistic UX patterns inspired by Amazon, eBay, and Shopify
- Fully responsive design for both desktop and mobile viewports

1.2 Tech Stack

Layer	Technology	Version	Purpose
Runtime	Node.js	18+	JavaScript server runtime
Framework	Express.js	4.x	REST API server
Database	MongoDB + Mongoose	8.x	Document store + ODM
Frontend	React	18.x	Component-based UI
Bundler	Vite	5.x	Fast dev server + build
Routing	React Router	v6	Client-side page routing
State	Context API + useReducer	built-in	Cart state management
HTTP	Axios	1.x	API calls from frontend
Env	dotenv	16.x	Environment variable loading

1.3 Key Features

Feature	Description
Product Browsing	Grid layout with images, ratings, prices, and stock badges
Category Filtering	Filter by Electronics, Books, Clothing, Home & Kitchen

Search	URL-based keyword search across name, category, description
Sort	Sort by Featured, Price (asc/desc), Highest Rated
Product Detail	Full info page with quantity selector and cart interaction
Shopping Cart	Add, remove, update qty; persisted via localStorage
Order Summary	Live subtotal, shipping threshold, estimated tax
Fake Checkout	3-step wizard (Shipping → Payment → Review) + success screen
Responsive Design	Mobile hamburger menu, adaptive grid, touch-friendly controls

02 SYSTEM ARCHITECTURE

System Architecture

2.1 Folder Structure

```
mini-ecommerce/ ■■■ backend/ ■ ■■■ config/ ■ ■ ■■■ db.js # MongoDB connection ■ ■■■ controllers/ ■ ■ ■■■ productController.js # Business logic (5 handlers) ■ ■■■ models/ ■ ■ ■■■ Product.js # Mongoose schema ■ ■■■ routes/ ■ ■ ■■■ productRoutes.js # Express route definitions ■ ■■■ seed/ ■ ■ ■■■ data.js # 12 sample products ■ ■ ■■■ seeder.js # Import / destroy script ■ ■■■ .env.example ■ ■■■ package.json ■ ■■■ server.js # Entry point ■ ■■■ frontend/ ■■■ public/ ■■■ src/ ■■■ components/ ■ ■■■ CartItem.jsx / .css ■ ■■■ CategoryFilter.jsx / .css ■ ■■■ ErrorMessage.jsx / .css ■ ■■■ Footer.jsx / .css ■ ■■■ LoadingSpinner.jsx / .css ■ ■■■ Navbar.jsx / .css ■ ■■■ ProductCard.jsx / .css ■■■ context/ ■ ■■■ CartContext.jsx # useReducer + localStorage ■■■ pages/ ■ ■■■ CartPage.jsx / .css ■ ■■■ CheckoutPage.jsx / .css ■ ■■■ HomePage.jsx / .css ■ ■■■ NotFoundPage.jsx / .css ■ ■■■ ProductPage.jsx / .css ■■■ services/ ■ ■■■ api.js # All Axios calls ■■■ App.jsx # Routes ■■■ index.css # Design tokens + reset ■■■ main.jsx # ReactDOM entry
```

2.2 Data Flow

The request lifecycle from browser to database and back:

```
Browser (React) Server (Node.js) HomePage / ProductPage Express Router ■ ■ services/api.js ■■■■ HTTP GET ■■■■■ routes/productRoutes.js (Axios) ■ ■ controllers/productController.js ■ ■ CartContext ■■■■■ JSON response ■■ models/Product.js (useReducer) ■ ■ MongoDB Atlas localStorage (products collection) (persistence)
```

2.3 Frontend Architecture Principles

- **Pages are containers** — fetch data, manage loading/error state, pass props down
- **Components are pure UI** — receive props, emit events, no direct API calls
- **services/api.js is the only Axios file** — all fetch logic centralized
- **CartContext owns all cart state** — components call addItem / removeItem / updateQuantity
- **localStorage sync via useEffect** — cart persists across page refreshes
- **URL-driven filtering** — ?search= and ?category= are shareable and history-friendly

03 BACKEND — STAGE 1

Backend — Stage 1

The backend is a Node.js/Express REST API connected to MongoDB via Mongoose. It follows an MVC pattern: models define data shape, controllers hold business logic, and routes wire HTTP verbs to controller functions.

3.1 Setup & Configuration

SERVER.JS — ENTRY POINT

```
require('dotenv').config(); const express = require('express'); const cors = require('cors');
const connectDB = require('./config/db'); const app = express(); connectDB(); // connect to
MongoDB app.use(cors({ origin: ['http://localhost:5173', 'http://localhost:3000'] }));
app.use(express.json()); // parse JSON bodies app.use('/api/products', productRoutes);
app.get('/api/health', (req, res) => res.json({ success: true })); app.listen(process.env.PORT
|| 5000);
```

.ENV FILE

```
PORT=5000 MONGO_URI=mongodb://localhost:27017/ecommerce NODE_ENV=development
```

3.2 Product Model

Mongoose schema for the Product collection:

Field	Type	Required	Notes
name	String	Yes	Product display name
price	Number	Yes	Must be >= 0
category	String	Yes	Electronics / Books / Clothing / Home & Kitchen
image	String	Yes	URL to product image
description	String	No	Optional product description
stock	Number	Yes	Integer >= 0; default 0
rating	Number	No	Float 0–5; default 0
reviewCount	Number	No	Integer; default 0
createdAt	Date	Auto	timestamps: true
updatedAt	Date	Auto	timestamps: true

3.3 REST API Endpoints

Method	Endpoint	Description	Query Params
GET	/api/health	Server health check	—
GET	/api/products	Return all products	?category=Electronics
GET	/api/products/categories	Return all unique categories	—
GET	/api/products/:id	Return single product by ID	—
POST	/api/products	Create a new product	—
DELETE	/api/products/:id	Delete product by ID	—

3.4 Response Envelope

All endpoints return a consistent JSON shape:

```
// Success (list) { "success": true, "count": 12, "data": [ ...products ] } // Success (single) { "success": true, "data": { ...product } } // Error { "success": false, "message": "Product not found" }
```

3.5 Seed Data

The seed file (seed/data.js) contains 12 pre-built products across 4 categories. Run the seeder with:

```
npm run seed # import all 12 products node seed/seeder.js --destroy # wipe the collection
```

Category	Products Included
Electronics	Wireless Headphones, Mechanical Keyboard, USB-C Hub 7-in-1
Books	Clean Code, The Pragmatic Programmer, JavaScript: The Good Parts
Clothing	Classic Fit T-Shirt, Slim-Fit Chino Pants
Home & Kitchen	French Press, Bamboo Cutting Board Set, LED Desk Lamp, Cast Iron Skillet

04 FRONTEND — STAGE 2

Frontend — Stage 2

Stage 2 establishes the Vite + React project, routing structure, API service layer, and the two core browsing pages: HomePage and ProductPage.

4.1 Routing

Path	Component	Description
/	HomePage	Product grid with category filter, search, sort
/product/:id	ProductPage	Full product detail with qty selector
/cart	CartPage	Cart item list + order summary
/checkout	CheckoutPage	3-step checkout wizard
*	NotFoundPage	404 fallback

4.2 API Service Layer (services/api.js)

All Axios calls live here. Components never call fetch() or axios directly — they always import from this module. This means changing the base URL or adding auth headers is a one-file change.

```
// services/api.js import axios from 'axios'; const api = axios.create({ baseURL:
import.meta.env.VITE_API_URL || '' }); export const fetchProducts = async (category = '') => {
... }; export const fetchCategories = async () => { ... }; export const fetchProductById =
async (id) => { ... };
```

In development, vite.config.js proxies all `/api/*` requests to `http://localhost:5000`, eliminating CORS issues during local development.

4.3 HomePage Features

- Fetches products and categories simultaneously with Promise.all()
- Category filter triggers a new API request (server-side filtering)
- Search is client-side, applied on top of the fetched list
- Sort (Featured / Price asc / Price desc / Highest Rated) is also client-side
- URL params ?search= and ?category= are reflected in state on mount
- Loading spinner and error box with retry shown during async states

05 CART SYSTEM — STAGE 3

Cart System — Stage 3

The cart is implemented as a React Context backed by useReducer. It is the single source of truth for all cart state, exposed via a custom hook.

5.1 CartContext Architecture

```
// context/CartContext.jsx Actions: ADD_ITEM | REMOVE_ITEM | UPDATE_QTY | CLEAR_CART State shape: items: [{ _id, name, price, image, category, stock, quantity }] Derived values (computed on every render): totalItems = sum of all quantities totalPrice = sum of (price * quantity) for each item localStorage key: 'shopswift_cart' • Loaded on first render via initializer function • Saved via useEffect on every items change
```

5.2 Public Cart API

Function	Signature	Behaviour
addItem	addItem(product, qty?)	Adds new item or increments qty; respects stock ceiling
removeItem	removeItem(id)	Removes item by _id
updateQuantity	updateQuantity(id, qty)	Sets qty; if qty <= 0, removes item
clearCart	clearCart()	Empties the cart (used after checkout)
isInCart	isInCart(id) → bool	Returns true if product already in cart
getItemQuantity	getItemQuantity(id) → num	Returns current qty for product (0 if absent)

5.3 CartPage Layout

The CartPage uses a two-column grid on desktop:

Left column (flex: 1)	Right column (340px sticky)
Column headers (Product / Qty / Subtotal)	Order Summary heading
CartItem rows (image, name, qty stepper, subtotal, remove)	Subtotal, Shipping, Tax rows
Continue Shopping link	Free shipping threshold message
	Order Total
	Proceed to Checkout button

On mobile (<960px) the columns stack vertically with the summary moved above the item list for immediate visibility of the total.

06 CHECKOUT & UI POLISH — STAGE 4

Checkout & UI Polish — Stage 4

6.1 Checkout Flow

CheckoutPage is a multi-step wizard with three steps managed by a single `step` state integer (0, 1, 2).

Step	Name	Fields Collected	Next Action
0	Shipping	First/Last name, Email, Address, City, ZIP, Country	Continue to Payment →
1	Payment	Card name, Card number, Expiry, CVV	Review Order →
2	Review	Read-only summary; edit links back to prior steps	Place Order (fake)
—	Success	Animated confirmation screen	clearCart() + home link

Simulated network delay: On "Place Order", a 1.2-second setTimeout runs before clearCart() is called and the success screen renders. A spinner inside the button provides feedback during the wait. No HTTP request is made — the checkout is entirely client-side.

6.2 Navbar Upgrades

- **Primary bar (64px)** — Logo, search field, cart icon with live badge, hamburger
- **Secondary bar (38px)** — Category shortcut links (Electronics, Books, Clothing, Home & Kitchen) + Deals
- **Mobile hamburger** — Animated 3-bar → X icon; slide-down panel with search + category links
- **Cart badge animation** — Scales in from 0 with @keyframes badgePop when count changes
- **Active category highlight** — bottom-border underline in accent color via NavLink activeClassName

6.3 Homepage Hero

- Dark background panel with radial gradient accent and dot-grid texture overlay
- Eyebrow pill badge (free shipping notice) above the heading
- Fraunces serif display heading with clamp() for fluid font-size scaling
- Three stat cards: Products / Categories / Avg Rating
- Sort dropdown added to toolbar alongside the category filter pills

07 API REFERENCE

API Reference

7.1 GET /api/products

URL	GET /api/products
Query	?category=Electronics (optional, case-insensitive regex match)
Returns	Array of all matching Product objects
Success	200 { success: true, count: N, data: [...] }
Error	500 { success: false, message: "..." }

7.2 GET /api/products/categories

URL	GET /api/products/categories
Query	None
Returns	Sorted array of unique category strings
Success	200 { success: true, data: ["Books","Clothing","Electronics","Home & Kitchen"] }

7.3 GET /api/products/:id

URL	GET /api/products/:id
Params	:id — MongoDB ObjectId string
Returns	Single Product object
Success	200 { success: true, data: { ...product } }
Not Found	404 { success: false, message: "Product not found" }
Bad ID	400 { success: false, message: "Invalid product ID format" }

7.4 POST /api/products

URL	POST /api/products
Body	JSON: { name, price, category, image, description?, stock, rating?, reviewCount? }
Returns	Created Product object

Success	201 { success: true, message: "Product created successfully", data: {...} }
Validation	400 { success: false, message: "Validation failed", errors: [...] }

7.5 DELETE /api/products/:id

URL	DELETE /api/products/:id
Params	:id — MongoDB ObjectId string
Returns	Deleted product ID
Success	200 { success: true, message: "Product deleted successfully", data: { id } }
Not Found	404 { success: false, message: "Product not found" }

08 COMPONENT REFERENCE

Component Reference

Navbar

`components/Navbar.jsx`

Sticky two-bar header. Uses useCart() for badge count. Manages searchQuery state and hamburger open/close.

Prop / Context	Type / Notes
No props	Reads from CartContext

Footer

`components/Footer.jsx`

Dark footer with brand mark and nav links.

Prop / Context	Type / Notes
No props	Pure presentational

ProductCard

`components/ProductCard.jsx`

Grid card displayed on HomePage. Calls addItem() on button click. Shows flash 'Added!' state for 1.5s. Prevents Link navigation on button click via e.preventDefault().

Prop / Context	Type / Notes
product	Product object from API

CartItem

`components/CartItem.jsx`

Single row in CartPage item list. Calls updateQuantity / removeItem from CartContext.

Prop / Context	Type / Notes
item	Cart item object { _id, name, price, image, category, stock, quantity }

CategoryFilter

`components/CategoryFilter.jsx`

Row of pill buttons for category selection. 'All' pill always first.

Prop / Context	Type / Notes
categories	string[]
selected	string — active category
onSelect	fn(category: string)

LoadingSpinner

`components/LoadingSpinner.jsx`

Centered spinner + message for async loading states.

Prop / Context	Type / Notes
message	string — default "Loading…"

ErrorMessage

`components/ErrorMessage.jsx`

Alert box with warning icon, message text, and optional retry button.

Prop / Context	Type / Notes
message	string — error text
onRetry	fn — optional callback for retry button

09 SETUP & RUNNING LOCALLY

Setup & Running Locally

9.1 Prerequisites

Tool	Version	Download
Node.js	18+	https://nodejs.org
npm	9+	Bundled with Node.js
MongoDB	6+	https://mongodb.com/try/download/community
Git	Any	https://git-scm.com

9.2 Backend Setup

```
# 1. Enter the backend directory cd backend # 2. Install dependencies npm install # 3. Create your environment file cp .env.example .env # Edit .env and set MONGO_URI to your MongoDB connection string # 4. Seed the database with sample products npm run seed # 5. Start the development server (with auto-reload) npm run dev # Server starts on http://localhost:5000 # Health check: GET http://localhost:5000/api/health
```

9.3 Frontend Setup

```
# 1. Enter the frontend directory (new terminal) cd frontend # 2. Install dependencies npm install # 3. Start Vite dev server npm run dev # App starts on http://localhost:5173 # API calls are automatically proxied to :5000
```

9.4 Environment Variables

Variable	Location	Default	Description
PORT	backend	5000	Express server port
MONGO_URI	backend	mongodb://localhost:27017/ecommerce	MongoDB connection string
NODE_ENV	backend	development	Environment mode
VITE_API_URL	frontend	"" (empty = proxy)	Override API base URL in production

9.5 Available npm Scripts

Directory	Command	Description
backend	npm run dev	Start with nodemon (auto-reload on save)
backend	npm start	Start without nodemon (production)
backend	npm run seed	Import seed products into MongoDB
backend	node seed/seeder --destroy	Wipe all products from MongoDB
frontend	npm run dev	Start Vite development server
frontend	npm run build	Build production bundle to /dist
frontend	npm run preview	Preview the production build locally

10 DESIGN SYSTEM

Design System

All design tokens are defined as CSS custom properties on :root in `src/index.css` and used consistently throughout every component and page stylesheet.

10.1 Color Palette

- **--col-text-1** #1a1714 — Primary text, navbar background, dark UI surfaces
- **--col-accent** #c84b31 — Primary accent — buttons, badges, active states
- **--col-accent-dk** #a33a22 — Accent hover / active state
- **--col-bg** #f7f5f0 — Page background — warm off-white
- **--col-surface** #ffffff — Card and panel surfaces
- **--col-border** #e8e4dd — Borders and dividers
- **--col-text-2** #5c5650 — Secondary text — descriptions, labels
- **--col-text-3** #9c958d — Tertiary text — placeholders, captions
- **--col-star** #e6a817 — Star rating fill color
- **--col-badge** #1a6b3a — Success badge (in-stock, added-to-cart)

10.2 Typography

Variable	Family	Usage
--font-display	Fraunces (Google Fonts)	Hero headings, prices, brand name — serif editorial
--font-body	DM Sans (Google Fonts)	All UI text, labels, buttons, paragraphs

10.3 Spacing Scale

Token	Value	Typical Use
--space-xs	4px	Icon gaps, star gaps
--space-sm	8px	Badge padding, small gaps
--space-md	16px	Card padding, field gaps, standard margin
--space-lg	24px	Section padding, grid gaps

--space-xl	40px	Page section top/bottom padding
--space-2xl	64px	Major page section spacing

10.4 Border Radii & Easing

Token	Value	Used For
--radius-sm	6px	Inputs, small buttons, image thumbnails
--radius-md	12px	Cards, modals, info panels
--radius-lg	20px	Large cards, checkout panels, image wrappers
--ease	cubic-bezier(0.22, 1, 0.36, 1)	All transitions — natural deceleration curve

ShopSwift — Academic E-Commerce Project · Built in 4 stages · Node.js + Express + MongoDB + React + Vite